Chapter 8. Suavização exponencial

▶ Carregar bibliotecas e configurações comuns

▶ Carregar bibliotecas adicionais

O alisamento exponencial foi proposto no final da década de 1950 (Brown, 1959 ; Holt, 1957 ; Winters, 1960) e motivou alguns dos métodos de previsão mais bem-sucedidos. As previsões geradas por meio de métodos de alisamento exponencial são médias ponderadas de observações passadas, com os pesos decaindo exponencialmente à medida que as observações se tornam mais antigas. Em outras palavras, quanto mais recente a observação, maior o peso associado. Essa estrutura gera previsões confiáveis rapidamente e para uma ampla gama de séries temporais, o que representa uma grande vantagem e de suma importância para aplicações na indústria.

Este capítulo está dividido em duas partes. Na primeira parte (Seções 8.1 a 8.4), apresentamos a mecânica dos métodos de suavização exponencial mais importantes e sua aplicação na previsão de séries temporais com diversas características. Isso nos ajuda a desenvolver uma intuição sobre como esses métodos funcionam. Nesse contexto, a seleção e o uso de um método de previsão podem parecer um tanto arbitrários. A escolha do método geralmente se baseia no reconhecimento dos principais componentes da série temporal (tendência e sazonalidade) e na forma como esses componentes entram no método de suavização (por exemplo, de maneira aditiva, amortecida ou multiplicativa).

Na segunda parte do capítulo (Seções 8.5 a 8.7), apresentamos os modelos estatísticos que fundamentam os métodos de suavização exponencial. Esses modelos geram previsões pontuais idênticas às dos métodos discutidos na primeira parte do capítulo, mas também geram intervalos de previsão. Além disso, essa estrutura estatística permite a seleção genuína de modelos entre modelos concorrentes.

8.1 Suavização exponencial simples

O método de suavização exponencial mais simples é naturalmente chamado de **suavização exponencial simples** (SES) [1] . Este método é adequado para prever dados sem tendência clara ou padrão sazonal. Por exemplo, os dados na Figura 8.1 não exibem nenhum comportamento de

tendência claro ou sazonalidade. (Há um declínio nos últimos anos, o que pode sugerir uma tendência. Consideraremos se um método baseado em tendência seria melhor para esta série mais adiante neste capítulo.) Já consideramos os métodos ingênuo e da média como possíveis métodos para prever tais dados (Seção 5.2).

```
algeria_economy = pd.read_csv("../data/algeria_exports.csv",
    parse_dates=["ds"])
plot_series(algeria_economy,
    xlabel="Year [1Y]", ylabel="% of GDP", title="Exports: Algeria")
```

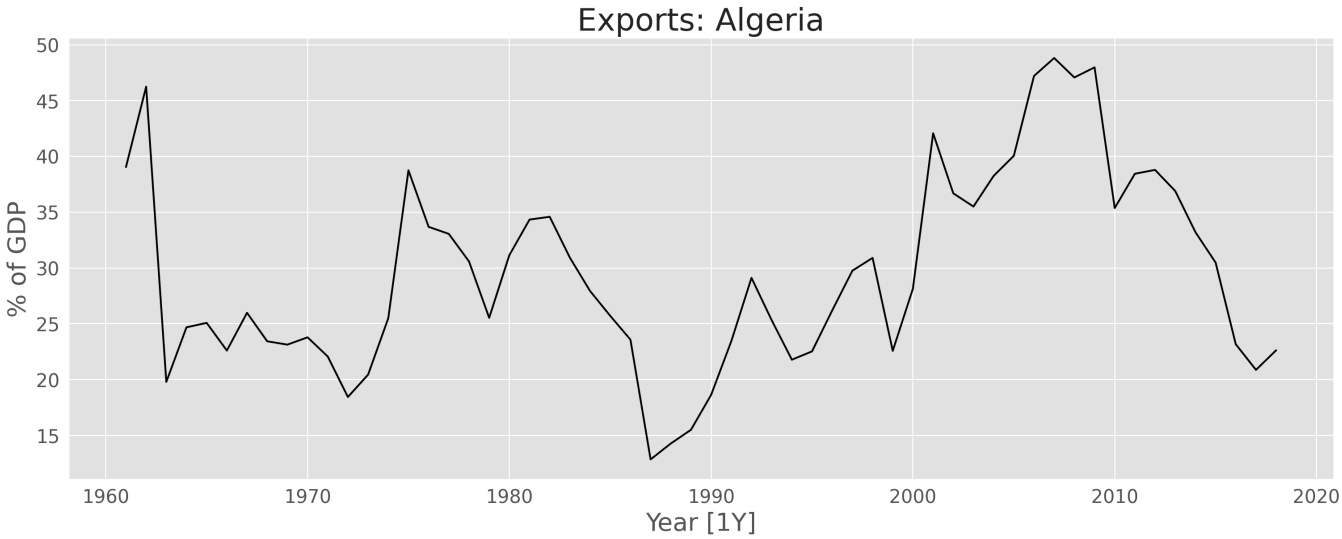

Figura 8.1: Exportações de bens e serviços da Argélia de 1960 a 2017.

Utilizando o método ingênuo, todas as previsões para o futuro são iguais ao último valor observado da série.

$$\hat{y}_{T+h|T} = y_T,$$

para$h = 1, 2, \ldots$.Portanto, o método ingênuo pressupõe que a observação mais recente é a única importante e que todas as observações anteriores não fornecem informações para o futuro. Isso pode ser interpretado como uma média ponderada, onde todo o peso é atribuído à última observação.

Utilizando o método da média, todas as previsões futuras são iguais à média simples dos dados observados.

$$\hat{y}_{T+h|T} = \frac{1}{T}\sum_{t=1}^{T} y_t,$$

para$h = 1, 2, \ldots$.Portanto, o método da média pressupõe que todas as observações têm a mesma importância e lhes atribui pesos iguais na geração de previsões.

We often want something between these two extremes. For example, it may be sensible to attach larger weights to more recent observations than to observations from the distant past.

This is exactly the concept behind simple exponential smoothing. Forecasts are calculated using weighted averages, where the weights decrease exponentially as observations come from further in the past — the smallest weights are associated with the oldest observations:

$$\hat{y}_{T+1|T} = \alpha y_T + \alpha(1-\alpha)y_{T-1} + \alpha(1-\alpha)^2 y_{T-2} + \cdots\,, \tag{8.1}$$

where $0 \leq \alpha \leq 1$ is the smoothing parameter. The one-step-ahead forecast for time $T+1$ is a weighted average of all of the observations in the series $y_1, \ldots, y_T$. The rate at which the weights decrease is controlled by the parameter α.

The table below shows the weights attached to observations for four different values of α when forecasting using simple exponential smoothing. Note that the sum of the weights even for a small value of α will be approximately one for any reasonable sample size.

	$\alpha = 0.2$	$\alpha = 0.4$	$\alpha = 0.6$	$\alpha = 0.8$
y_T	0.2000	0.4000	0.6000	0.8000
y_{T-1}	0.1600	0.2400	0.2400	0.1600
y_{T-2}	0.1280	0.1440	0.0960	0.0320
y_{T-3}	0.1024	0.0864	0.0384	0.0064
y_{T-4}	0.0819	0.0518	0.0154	0.0013
y_{T-5}	0.0655	0.0311	0.0061	0.0003

For any α between 0 and 1, the weights attached to the observations decrease exponentially as we go back in time, hence the name "exponential smoothing". If α is small (i.e., close to 0), more weight is given to observations from the more distant past. If α is large (i.e., close to 1), more weight is given to the more recent observations. For the extreme case where $\alpha = 1$, $\hat{y}_{T+1|T} = y_T$, and the forecasts are equal to the naïve forecasts.

We present two equivalent forms of simple exponential smoothing, each of which leads to the forecast Equation 8.1.

Weighted average form

The forecast at time $T+1$ is equal to a weighted average between the most recent observation y_T and the previous forecast $\hat{y}_{T|T-1}$:

$$\hat{y}_{T+1|T} = \alpha y_T + (1-\alpha)\hat{y}_{T|T-1},$$

where $0 \leq \alpha \leq 1$ is the smoothing parameter. Similarly, we can write the fitted values as

$$\hat{y}_{t+1|t} = \alpha y_t + (1-\alpha)\hat{y}_{t|t-1},$$

for $t = 1, \ldots, T$. (Recall that fitted values are simply one-step forecasts of the training data.)

The process has to start somewhere, so we let the first fitted value at time 1 be denoted by ℓ_0 (which we will have to estimate). Then

$$\begin{aligned}
\hat{y}_{2|1} &= \alpha y_1 + (1-\alpha)\ell_0 \\
\hat{y}_{3|2} &= \alpha y_2 + (1-\alpha)\hat{y}_{2|1} \\
\hat{y}_{4|3} &= \alpha y_3 + (1-\alpha)\hat{y}_{3|2} \\
&\vdots \\
\hat{y}_{T|T-1} &= \alpha y_{T-1} + (1-\alpha)\hat{y}_{T-1|T-2} \\
\hat{y}_{T+1|T} &= \alpha y_T + (1-\alpha)\hat{y}_{T|T-1}.
\end{aligned}$$

Substituting each equation into the following equation, we obtain

$$\begin{aligned}
\hat{y}_{3|2} &= \alpha y_2 + (1-\alpha)\left[\alpha y_1 + (1-\alpha)\ell_0\right] \\
&= \alpha y_2 + \alpha(1-\alpha)y_1 + (1-\alpha)^2\ell_0 \\
\hat{y}_{4|3} &= \alpha y_3 + (1-\alpha)[\alpha y_2 + \alpha(1-\alpha)y_1 + (1-\alpha)^2\ell_0] \\
&= \alpha y_3 + \alpha(1-\alpha)y_2 + \alpha(1-\alpha)^2 y_1 + (1-\alpha)^3\ell_0 \\
&\vdots \\
\hat{y}_{T+1|T} &= \sum_{j=0}^{T-1} \alpha(1-\alpha)^j y_{T-j} + (1-\alpha)^T\ell_0.
\end{aligned}$$

The last term becomes tiny for large T. So, the weighted average form leads to the same forecast Equation 8.1.

Component form

An alternative representation is the component form. For simple exponential smoothing, the only component included is the level, ℓ_t. (Other methods which are considered later in this chapter may also include a trend b_t and a seasonal component s_t.) Component form representations of exponential smoothing methods comprise a forecast equation and a smoothing equation for each of the components included in the method. The component form of simple exponential smoothing is given by:

$$\begin{aligned}
\text{Forecast equation} \quad & \hat{y}_{t+h|t} = \ell_t \\
\text{Smoothing equation} \quad & \ell_t = \alpha y_t + (1-\alpha)\ell_{t-1},
\end{aligned}$$

where ℓ_t is the level (or the smoothed value) of the series at time t. Setting $h = 1$ gives the fitted values, while setting $t = T$ gives the true forecasts beyond the training data.

The forecast equation shows that the forecast value at time $t + 1$ is the estimated level at time t. The smoothing equation for the level (usually referred to as the level equation) gives the estimated level of the series at each period t.

If we replace ℓ_t with $\hat{y}_{t+1|t}$ and ℓ_{t-1} with $\hat{y}_{t|t-1}$ in the smoothing equation, we will recover the weighted average form of simple exponential smoothing.

The component form of simple exponential smoothing is not particularly useful on its own, but it will be the easiest form to use when we start adding other components.

Flat forecasts

Simple exponential smoothing has a "flat" forecast function:

$$\hat{y}_{T+h|T} = \hat{y}_{T+1|T} = \ell_T, \qquad h = 2, 3, \dots .$$

That is, all forecasts take the same value, equal to the last level component. Remember that these forecasts will only be suitable if the time series has no trend or seasonal component.

Optimisation

The application of every exponential smoothing method requires the smoothing parameters and the initial values to be chosen. In particular, for simple exponential smoothing, we need to select the values of α and ℓ_0. All forecasts can be computed from the data once we know those values. For the methods that follow there is usually more than one smoothing parameter and more than one initial component to be chosen.

In some cases, the smoothing parameters may be chosen in a subjective manner — the forecaster specifies the value of the smoothing parameters based on previous experience. However, a more reliable and objective way to obtain values for the unknown parameters is to estimate them from the observed data.

In Section 7.2, we estimated the coefficients of a regression model by minimising the sum of the squared residuals (usually known as SSE or "sum of squared errors"). Similarly, the unknown parameters and the initial values for any exponential smoothing method can be estimated by minimising the SSE. The residuals are specified as $e_t = y_t - \hat{y}_{t|t-1}$ for $t = 1, \dots, T$. Hence, we find the values of the unknown parameters and the initial values that minimise

$$\text{SSE} = \sum_{t=1}^{T} (y_t - \hat{y}_{t|t-1})^2 = \sum_{t=1}^{T} e_t^2. \tag{8.2}$$

Unlike the regression case (where we have formulas which return the values of the regression coefficients that minimise the SSE), this involves a non-linear minimisation problem, and we need to use an optimisation tool to solve it.

Example: Algerian exports

In this example, simple exponential smoothing is applied to forecast exports of goods and services from Algeria. As we will explain later, simple exponential smoothing is a specific instance of the `AutoETS()` model.

```
sf = StatsForecast(freq="Y",
    models=[AutoETS(model="ANN", alias="SES")])
```

To generate the forecasts, you can use the `forecast()` method from the `StatsForecast` class. Setting `fitted=True` will save the fitted values so that they can be accessed later through the `forecast_fitted_values`.

```
fc = sf.forecast(df=algeria_economy, h=5, level=[80, 95], fitted=True)
fc.head()
```

	unique_id	ds	SES	SES-lo-95	SES-lo-80	SES-hi-80	SES-hi-95
0	Algeria	2018-12-31	22.4	10.5	14.6	30.2	34.4
1	Algeria	2019-12-31	22.4	6.9	12.3	32.6	38.0
2	Algeria	2020-12-31	22.4	3.9	10.3	34.5	41.0
3	Algeria	2021-12-31	22.4	1.4	8.7	36.2	43.5
4	Algeria	2022-12-31	22.4	-0.9	7.2	37.7	45.7

```
fitted_vals = sf.forecast_fitted_values()
fitted_vals.head()
```

	unique_id	ds	y	SES	SES-lo-95	SES-lo-80	SES-hi-80	SES-hi-95
0	Algeria	1960-12-31	39.0	39.5	27.7	31.8	47.3	51.3
1	Algeria	1961-12-31	46.2	39.1	27.3	31.4	46.8	50.9
2	Algeria	1962-12-31	19.8	45.1	33.3	37.4	52.8	56.9
3	Algeria	1963-12-31	24.7	23.8	12.0	16.1	31.6	35.7
4	Algeria	1964-12-31	25.1	24.5	12.7	16.8	32.3	36.4

The `forecast()` method of the `StatsForecast` class is optimized for speed and scalability, so it does not store the estimated model parameters. Those can be accessed after using the `.fit()` or `.fit_predict()` method:

```
# Save model fit(s)
sf.fit(algeria_economy)
# Save model fit(s) and return forecasts
fc = sf.fit_predict(df=algeria_economy, h=5, level=[80, 95])
params = sf.fitted_[0, 0].model_["fit"].x
alpha, level_0 = np.round(params, 4)
```

Alternatively, you can instantiate and fit the model directly, as shown below.

```
ses = AutoETS(model="ANN", alias="SES")
ses.fit(y=algeria_economy["y"].values)
params = ses.model_["fit"].x
alpha, level_0 = np.round(params, 4)
print("Optimal parameters:")
print("alpha:", alpha)
print("Initial level:", level_0)
```

```
Optimal parameters:
alpha: 0.8401
Initial level: 39.53
```

The optimisation gives parameter estimates $\hat{\alpha} = 0.84$ and $\hat{\ell}_0 = 39.5$, obtained by minimising SSE over periods $t = 1, 2, \ldots, 58$, subject to the restriction that $0 \leq \alpha \leq 1$.

In Table 8.1 we demonstrate the calculation using these parameters. The second last column shows the estimated level for times $t = 0$ to $t = 58$; the last few rows of the last column show the forecasts for $h = 1$ to 5-steps ahead.

Table 8.1: Forecasting goods and services exports from Algeria using simple exponential smoothing.

Year	Time	Observation	Level	Forecast
	t	y_t	l_t	$\hat{y}_{t\|t-1}$
1959	0		39.54	
1960	1	39.04	39.12	39.54
1961	2	46.24	45.10	39.12
1962	3	19.79	23.84	45.10
1963	4	24.68	24.55	23.84
	$\vdots$	$\vdots$	$\vdots$	$\vdots$
2014	55	30.22	30.80	33.85
2015	56	23.17	24.39	30.80
2016	57	20.86	21.43	24.39
2017	58	22.64	22.44	21.43
	h			$\hat{y}_{T+h\|T}$
2018	1			22.44
2019	2			22.44
2020	3			22.44
2021	4			22.44

Year	Time	Observation	Level	Forecast
2022	5			22.44

```
fig = plot_series(algeria_economy, fc, level=[80, 95],
    xlabel="Year [1Y]", ylabel="% of GDP", title="Exports: Algeria",
    palette="black_and_blue", rm_legend=False)
fig.axes[0].plot("ds", "SES", data=fitted_vals, color="#D55E00")
fig
```

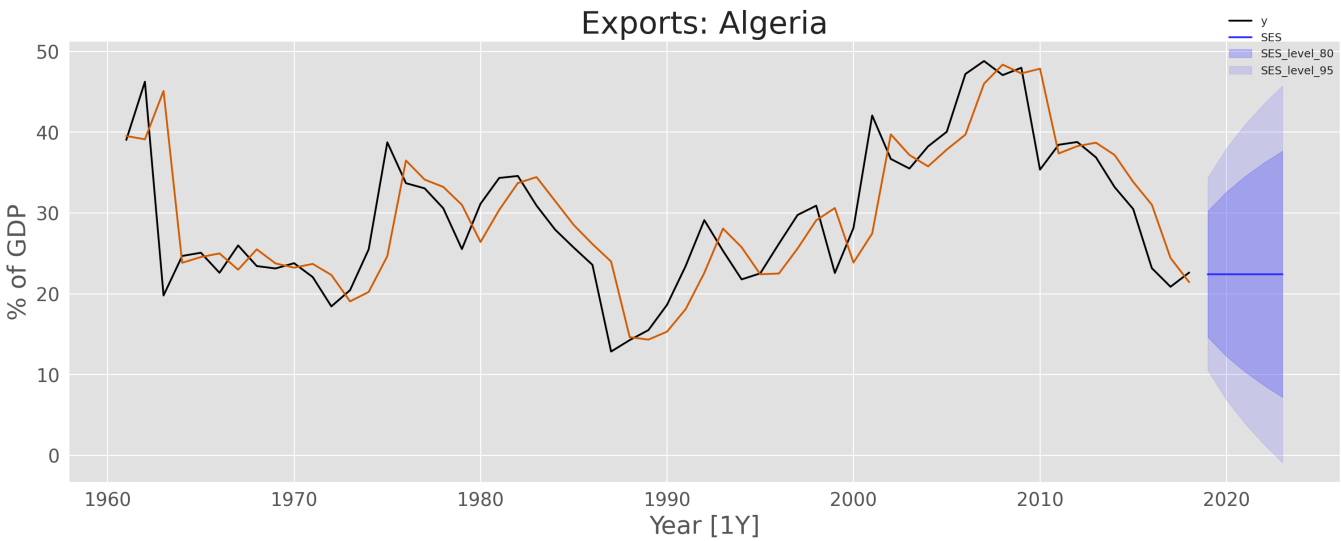

Figure 8.2: Simple exponential smoothing applied to exports from Algeria (1960–2017).

The forecasts for the period 2018–2022 are plotted in Figure 8.2. Also plotted are one-step-ahead fitted values alongside the data over the period 1960–2017. The large value of α in this example is reflected in the large adjustment that takes place in the estimated level ℓ_t at each time. A smaller value of α would lead to smaller changes over time, and so the series of fitted values would be smoother.

The prediction intervals shown here are calculated using the methods described in Section 8.7. The prediction intervals show that there is considerable uncertainty in the future exports over the five-year forecast period. So interpreting the point forecasts without accounting for the large uncertainty can be very misleading.

8.2 Methods with trend

Holt's linear trend method

Holt (1957) extended simple exponential smoothing to allow the forecasting of data with a trend. This method involves a forecast equation and two smoothing equations (one for the level

and one for the trend):

$$\begin{aligned} \text{Forecast equation} \quad \hat{y}_{t+h|t} &= \ell_t + hb_t \\ \text{Level equation} \quad \ell_t &= \alpha y_t + (1-\alpha)(\ell_{t-1} + b_{t-1}) \\ \text{Trend equation} \quad b_t &= \beta^*(\ell_t - \ell_{t-1}) + (1-\beta^*)b_{t-1}, \end{aligned}$$

where ℓ_t denotes an estimate of the level of the series at time t, b_t denotes an estimate of the trend (slope) of the series at time t, α is the smoothing parameter for the level, $0 \leq \alpha \leq 1$, and β^* is the smoothing parameter for the trend, $0 \leq \beta^* \leq 1$. (We denote this as β^* instead of β for reasons that will be explained in Section 8.5.)

As with simple exponential smoothing, the level equation here shows that ℓ_t is a weighted average of observation y_t and the one-step-ahead training forecast for time t, here given by $\ell_{t-1} + b_{t-1}$. The trend equation shows that b_t is a weighted average of the estimated trend at time t based on $\ell_t - \ell_{t-1}$ and b_{t-1}, the previous estimate of the trend.

The forecast function is no longer flat but trending. The h-step-ahead forecast is equal to the last estimated level plus h times the last estimated trend value. Hence the forecasts are a linear function of h.

Example: Australian population

```
aus_economy = (
    pd.read_csv("../data/aus_economy.csv", parse_dates=["ds"])
    .assign(y=lambda x: x["y"] / 1e6)
)
plot_series(aus_economy, xlabel="Year [1Y]", ylabel="Millions",
    title="Australian population")
```

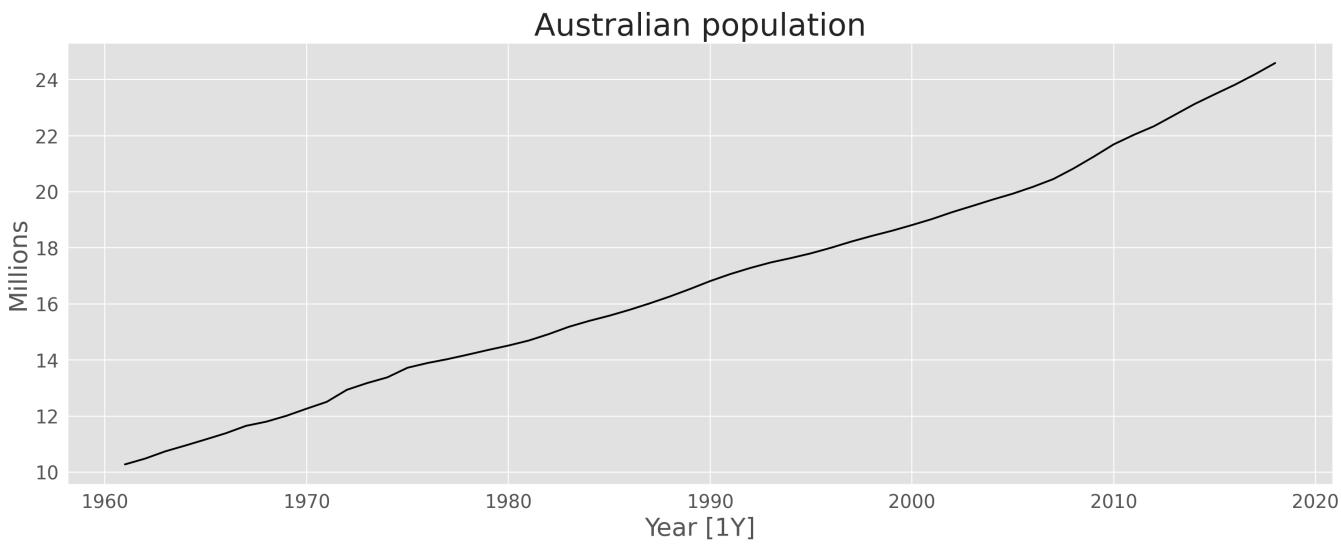

Figure 8.3: Australia's population, 1960-2017.

Figure 8.3 shows Australia's annual population from 1960 to 2017. We will apply Holt's method to this series. The smoothing parameters, α and β^*, and the initial values ℓ_0 and b_0 are estimated

by minimising the SSE for the one-step training errors as in Section 8.1.

```
sf = StatsForecast(models=[AutoETS(model="AAN", alias="Holt")], freq="A")
fc = sf.fit_predict(df=aus_economy, h=10)
```

You can also use `Holt` from `statsforecast.models`, which is a wrapper for this particular case of the `AutoETS()` class. As before, we used the `.fit_predict()` method in order to access the optimal parameters.

```
holt = sf.fitted_[0, 0]
params = np.round(holt.model_["fit"].x, 4)
print("Optimal parameters:")
print("alpha:", params[0])
print("beta:", params[1])
print("Initial level:", params[2])
print("Initial trend:", params[3])
```

```
Optimal parameters:
alpha: 0.9999
beta: 0.2001
Initial level: 10.0509
Initial trend: 0.2264
```

The estimated smoothing coefficient for the level is $\hat{\alpha} = 0.9999$. The very high value shows that the level changes rapidly in order to capture the highly trended series. The estimated smoothing coefficient for the slope is $\hat{\beta}^* = 0.20006$. This is relatively large suggesting that the trend also changes often (even if the changes are slight).

In Table 8.2 we use these values to demonstrate the application of Holt's method.

Table 8.2: Forecasting Australian annual population using Holt's linear trend method.

Year	Time	Observation	Level	Slope	Forecast
	t	y_t	l_t		$\hat{y}_{t+1\|t}$
1959	0		10.05	0.22	
1960	1	10.27	10.27	0.22	10.50
1961	2	10.48	10.48	0.22	10.70
1962	3	10.74	10.74	0.23	10.97
1963	4	10.95	10.95	0.22	11.17
1964	5	11.16	11.16	0.22	11.39
	⋮	⋮	⋮	⋮	⋮
2014	55	23.47	23.47	0.35	23.52

Year	Time	Observation	Level	Slope	Forecast
2015	56	23.81	23.81	0.35	23.87
2016	57	24.19	24.19	0.35	24.21
2017	58	24.69	24.59	0.36	24.57
	h				$\hat{y}_{T+h\|T}$
2018	1				24.95
2019	2				25.31
2020	3				25.67
2021	4				26.04
2022	5				26.40
2023	6				26.76
2024	7				27.12
2025	8				27.49
2026	9				27.85
2027	10				28.21

Damped trend methods

The forecasts generated by Holt's linear method display a constant trend (increasing or decreasing) indefinitely into the future. Empirical evidence indicates that these methods tend to over-forecast, especially for longer forecast horizons. Motivated by this observation, Gardner and McKenzie (1985) introduced a parameter that "dampens" the trend to a flat line some time in the future. Methods that include a damped trend have proven to be very successful, and are arguably the most popular individual methods when forecasts are required automatically for many series.

In conjunction with the smoothing parameters α and β^* (with values between 0 and 1 as in Holt's method), this method also includes a damping parameter $0 < \phi < 1$:

$$\begin{aligned}
\hat{y}_{t+h|t} &= \ell_t + (\phi + \phi^2 + \dots + \phi^h)b_t \\
\ell_t &= \alpha y_t + (1-\alpha)(\ell_{t-1} + \phi b_{t-1}) \\
b_t &= \beta^*(\ell_t - \ell_{t-1}) + (1-\beta^*)\phi b_{t-1}.
\end{aligned}$$

If $\phi = 1$, the method is identical to Holt's linear method. For values between 0 and 1, ϕ dampens the trend so that it approaches a constant some time in the future. In fact, the forecasts converge to $\ell_T + \phi b_T/(1-\phi)$ as $h \to \infty$ for any value $0 < \phi < 1$. This means that short-run forecasts are trended while long-run forecasts are constant.

In practice, ϕ is rarely less than 0.8 as the damping has a very strong effect for smaller values. Values of ϕ close to 1 will mean that a damped model is not able to be distinguished from a non-damped model. For these reasons, we usually restrict ϕ to a minimum of 0.8 and a maximum of 0.98.

Example: Australian Population (continued)

Figure 8.4 shows the forecasts for years 2018–2032 generated from Holt's linear trend method and the damped trend method.

```
sf = StatsForecast(
    models=[
        AutoETS(model="AAN", alias="Holt"),
        AutoETS(model="AAN", alias="Damped Holt's method",
            damped=True, phi=0.9),
    ],
    freq="Y",
)
fc = sf.forecast(df=aus_economy, h=15)
plot_series(aus_economy, fc,
    xlabel="Year [1Y]", ylabel="Millions", title="Australian population",
    palette="black_and_2color", rm_legend=False)
```

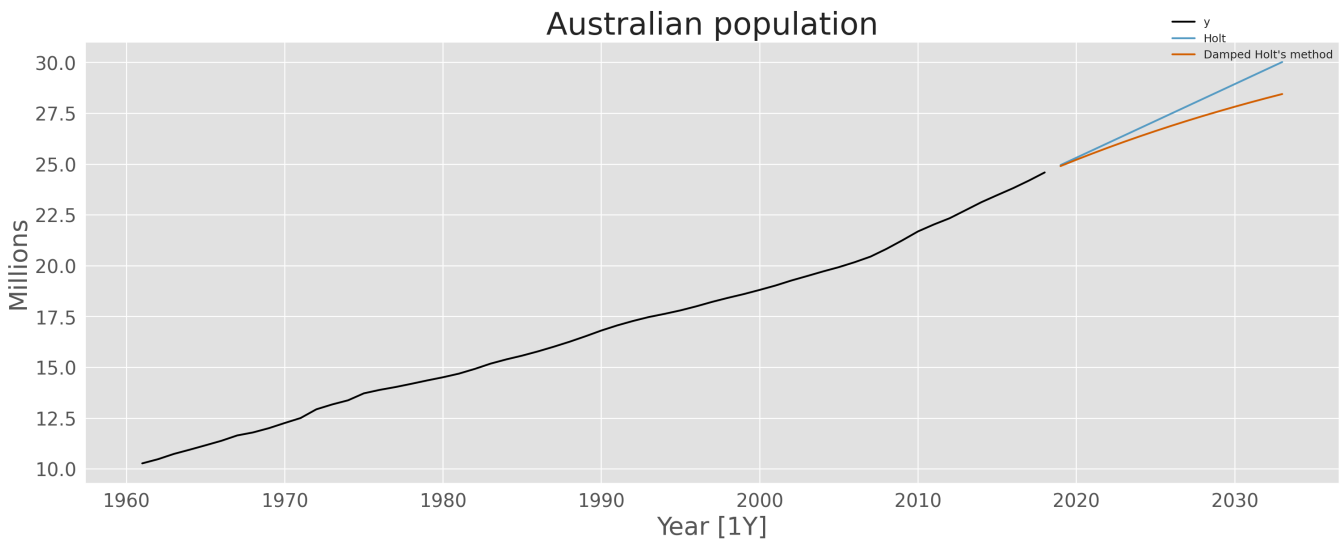

Figure 8.4: Forecasting annual Australian population (millions) over 2018-2032. For the damped trend method, $\phi = 0.90$.

We have set the damping parameter to a relatively low number ($\phi = 0.90$) to exaggerate the effect of damping for comparison. Usually, we would estimate ϕ along with the other parameters. We have also used a rather large forecast horizon ($h = 15$) to highlight the difference between a damped trend and a linear trend.

Example: Internet usage

In this example, we compare the forecasting performance of the three exponential smoothing methods that we have considered so far in forecasting the number of users connected to the

internet via a server. The data is observed over 100 minutes and is shown in Figure 8.5.

```
www_usage = pd.read_csv("../data/www_usage.csv")
plot_series(www_usage, xlabel="Minute", ylabel="Number of users",
    title="Internet usage per minute")
```

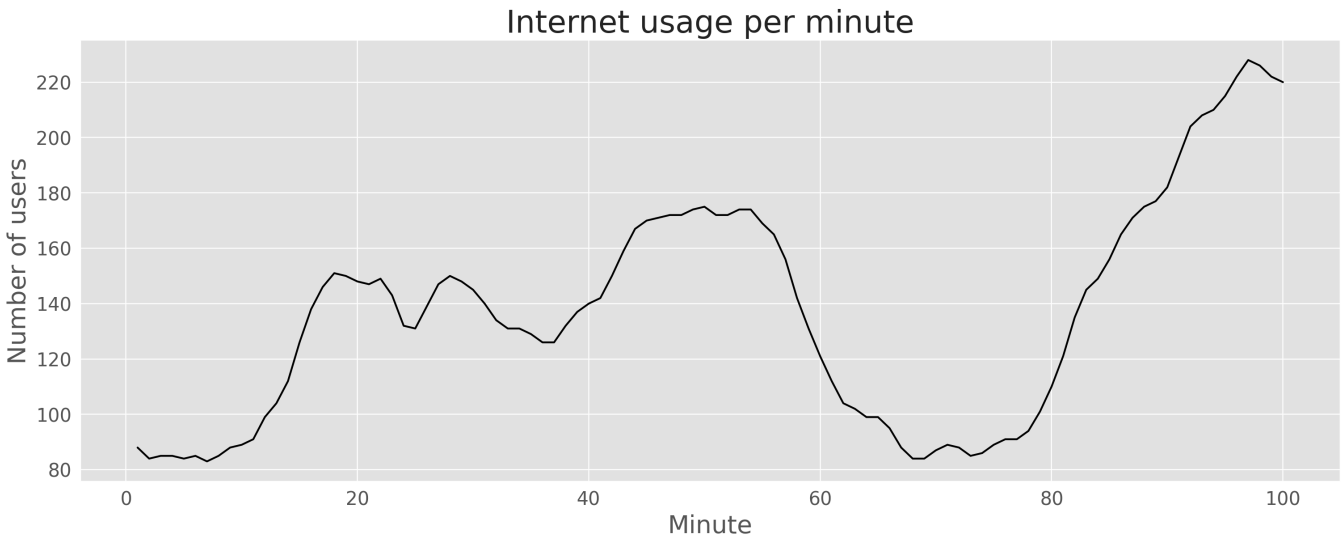

Figure 8.5: Users connected to the internet through a server.

We will use time series cross-validation to compare the one-step forecast accuracy of the three methods. We have set `step_size=1` to get the one-step forecasts and `n_windows=90` to cover most of the data.

```
from utilsforecast.losses import mape as _mape
def mape(df, models, id_col = "unique_id", target_col = "y"):
    """MAPE scaled to percentage."""
    df_mape = _mape(df, models, id_col=id_col, target_col=target_col)
    df_mape.loc[:, df_mape.select_dtypes(include='number').columns] *= 100
    return df_mape

sf = StatsForecast(
    models=[
        AutoETS(model="ANN", alias="SES"),
        AutoETS(model="AAN", alias="Holt"),
        AutoETS(model="AAN", damped=True, phi=0.9, alias="Damped"),
    ],
    freq=1,
)
cv = sf.cross_validation(df=www_usage, h=1, n_windows=90, step_size=1)
result_df = evaluate(
    cv.drop("cutoff", axis=1),
    metrics=[rmse, mae, mape, partial(mase, seasonality=1)],
    train_df=www_usage,
)
result_df.drop(columns="unique_id")
```

	metric	SES	Holt	Damped
0	rmse	6.07	5.09	5.04
1	mae	4.85	4.18	4.06
2	mape	3.59	3.16	3.06
3	mase	1.07	0.92	0.90

Damped Holt's method is best whether you compare RMSE, MAE, MAPE, or MASE values. So we will proceed with using the damped Holt's method and apply it to the whole data set to get forecasts for future minutes.

```
sf = StatsForecast(
    models=[AutoETS(model="AAN", damped=True, alias="Damped")],
    freq=1,
)
sf.fit(www_usage)
params = np.round(sf.fitted_[0, 0].model_["fit"].x, 4)
print("Optimal parameters:")
print("Damped alpha:", params[0])
print("Damped beta:", params[1])
print("Damped phi:", params[2])
print("Initial level:", params[3])
print("Initial trend:", params[4])
```

```
Optimal parameters:
Damped alpha: 0.9999
Damped beta: 0.2
Damped phi: 0.9087
Initial level: 88.8688
Initial trend: -0.9231
```

The smoothing parameter for the slope is estimated to be 0.2, indicating that the trend updates smoothly instead of reacting strongly to immediate changes in the last minutes of internet usage. The value of α is very close to one, indicating that the level component reacts strongly to each new observation.

```
level = [80, 95]
fc = sf.predict(h=10, level=level)
plot_series(www_usage, fc, level=level,
    xlabel="Minute", ylabel="Number of users",
    title="Internet usage per minute",
    palette="black_and_blue", rm_legend=False)
```

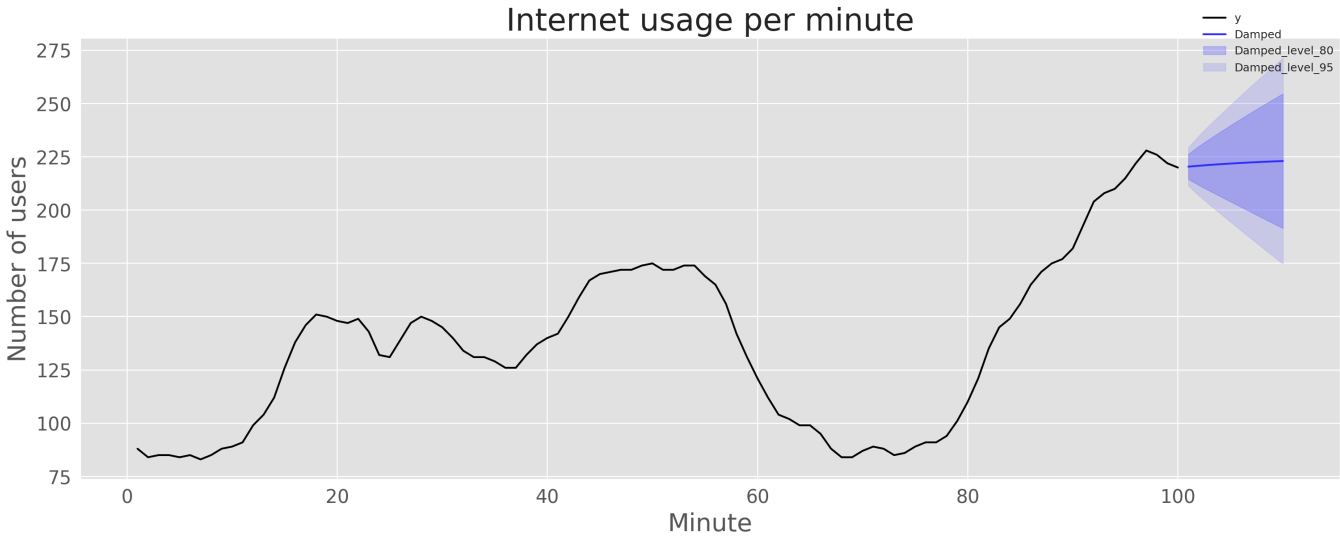

Figure 8.6: Forecasting internet usage: comparing forecasting performance of non-seasonal methods.

The resulting forecasts look sensible with a minimally increasing trend, and relatively wide prediction intervals reflecting the variation in the historical data. The prediction intervals are calculated using the methods described in Section 8.7.

In this example, the process of selecting a method was relatively easy as both MAE and RMSE comparisons suggested the same method (damped Holt's). However, sometimes different accuracy measures will suggest different forecasting methods, and then a decision is required as to which forecasting method we prefer to use. As forecasting tasks can vary by many dimensions (length of forecast horizon, size of test set, forecast error measures, frequency of data, etc.), it is unlikely that one method will be better than all others for all forecasting scenarios. What we require from a forecasting method are consistently sensible forecasts, and these should be frequently evaluated against the task at hand.

8.3 Methods with seasonality

Holt (1957) and Winters (1960) extended Holt's method to capture seasonality. The Holt-Winters seasonal method comprises the forecast equation and three smoothing equations — one for the level ℓ_t, one for the trend b_t, and one for the seasonal component s_t, with corresponding smoothing parameters α, β^* and γ. We use m to denote the period of the seasonality, i.e., the number of seasons in a year. For example, for quarterly data $m = 4$, and for monthly data $m = 12$.

There are two variations to this method that differ in the nature of the seasonal component. The additive method is preferred when the seasonal variations are roughly constant through the series, while the multiplicative method is preferred when the seasonal variations are changing proportional to the level of the series. With the additive method, the seasonal component is

expressed in absolute terms in the scale of the observed series, and in the level equation the series is seasonally adjusted by subtracting the seasonal component. Within each year, the seasonal component will add up to approximately zero. With the multiplicative method, the seasonal component is expressed in relative terms (percentages), and the series is seasonally adjusted by dividing through by the seasonal component. Within each year, the seasonal component will sum up to approximately m.

Holt-Winters' additive method

The component form for the additive method is:

$$\begin{aligned}\hat{y}_{t+h|t} &= \ell_t + hb_t + s_{t+h-m(k+1)}\\ \ell_t &= \alpha(y_t - s_{t-m}) + (1-\alpha)(\ell_{t-1} + b_{t-1})\\ b_t &= \beta^*(\ell_t - \ell_{t-1}) + (1-\beta^*)b_{t-1}\\ s_t &= \gamma(y_t - \ell_{t-1} - b_{t-1}) + (1-\gamma)s_{t-m},\end{aligned}$$

where k is the integer part of $(h-1)/m$, which ensures that the estimates of the seasonal indices used for forecasting come from the final year of the sample. The level equation shows a weighted average between the seasonally adjusted observation $(y_t - s_{t-m})$ and the non-seasonal forecast $(\ell_{t-1} + b_{t-1})$ for time t. The trend equation is identical to Holt's linear method. The seasonal equation shows a weighted average between the current seasonal index, $(y_t - \ell_{t-1} - b_{t-1})$, and the seasonal index of the same season last year (i.e., m time periods ago).

The equation for the seasonal component is often expressed as

$$s_t = \gamma^*(y_t - \ell_t) + (1-\gamma^*)s_{t-m}.$$

If we substitute ℓ_t from the smoothing equation for the level of the component form above, we get

$$s_t = \gamma^*(1-\alpha)(y_t - \ell_{t-1} - b_{t-1}) + [1 - \gamma^*(1-\alpha)]s_{t-m},$$

which is identical to the smoothing equation for the seasonal component we specify here, with $\gamma = \gamma^*(1-\alpha)$. The usual parameter restriction is $0 \leq \gamma^* \leq 1$, which translates to $0 \leq \gamma \leq 1-\alpha$.

Holt-Winters' multiplicative method

The component form for the multiplicative method is:

$$\begin{aligned}\hat{y}_{t+h|t} &= (\ell_t + hb_t)s_{t+h-m(k+1)}\\ \ell_t &= \alpha\frac{y_t}{s_{t-m}} + (1-\alpha)(\ell_{t-1} + b_{t-1})\\ b_t &= \beta^*(\ell_t - \ell_{t-1}) + (1-\beta^*)b_{t-1}\\ s_t &= \gamma\frac{y_t}{(\ell_{t-1} + b_{t-1})} + (1-\gamma)s_{t-m}.\end{aligned}$$

Example: Domestic overnight trips in Australia

We apply Holt-Winters' method with both additive and multiplicative seasonality[2] to forecast quarterly visitor nights in Australia spent by domestic tourists. Figure 8.7 shows the data from 1998–2017, and the forecasts for 2018–2020. The data show an obvious seasonal pattern, with peaks observed in the March quarter of each year, corresponding to the Australian summer.

```
tourism = pd.read_csv("../data/tourism.csv", parse_dates=["ds"])
aus_holidays = (
    tourism.loc[lambda x: x["Purpose"] == "Holiday"]
    .rename(columns={"Purpose": "unique_id"})
    .groupby(["unique_id", "ds"])
    ["y"].sum().div(1e3)
    .reset_index()
)
sf = StatsForecast(
    models=[
        AutoETS(season_length=4, model="AAA", alias="additive"),
        AutoETS(season_length=4, model="MAM", alias="multiplicative"),
    ],
    freq="Q",
)
fc = sf.forecast(df=aus_holidays, h=12, fitted=True)
plot_series(aus_holidays, fc,
    xlabel="Quarter", ylabel="Overnight trips (millions)",
    title="Australian domestic tourism",
    palette="black_and_2color", rm_legend=False)
```

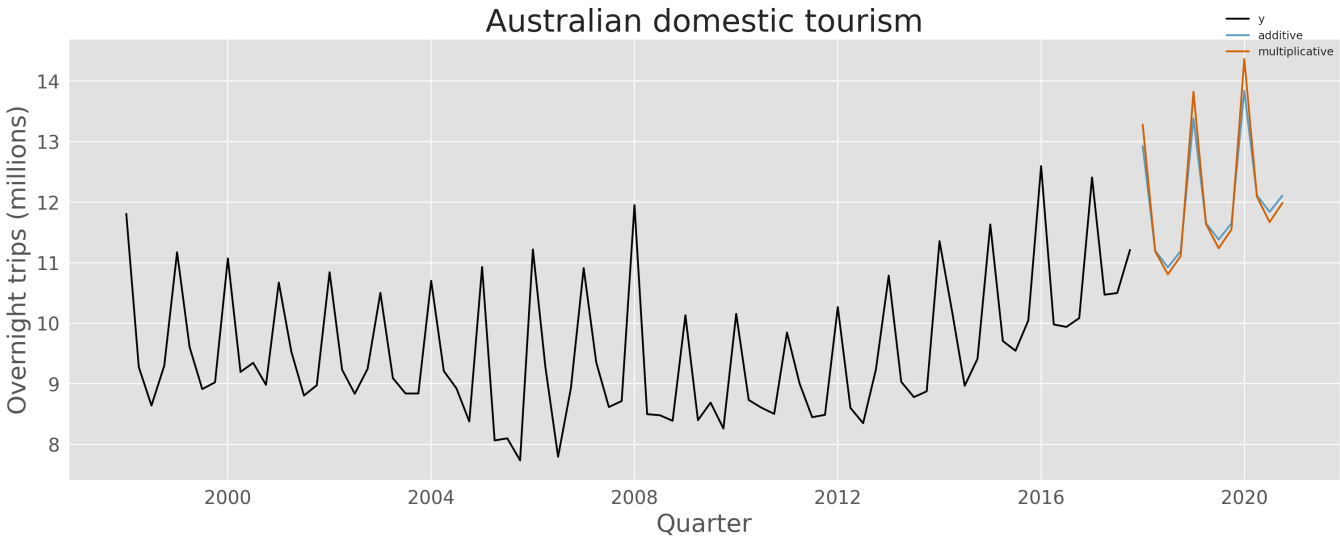

Figure 8.7: Forecasting domestic overnight trips in Australia using the Holt-Winters method with both additive and multiplicative seasonality.

You can also use `HoltWinters`, which is a wrapper for these particular cases of the `AutoETS()` class. Simply specify the `error_type` as either `A` (additive) or `M` (multiplicative).

```
HoltWinters(season_length=4, error_type="A", alias="additive")
HoltWinters(season_length=4, error_type="M", alias="multiplicative")
```

The applications of both methods (with additive and multiplicative seasonality) are presented in Table 8.3 and Table 8.4 respectively.

Table 8.3: Applying Holt-Winters' method with additive seasonality for forecasting domestic tourism in Australia. Notice that the additive seasonal component sums to approximately zero. The smoothing parameters are $\alpha = 0.2325$, $\beta^* = 0.03$, $\gamma = 0.0001$ and $\text{RMSE} = 0.412$.

Quarter	Time	Observation	Level	Slope	Season	Forecast
	t	y_t	l_t	b_t	s_t	$\hat{y}_{t+1\|t}$
1997 Q1	0				1.5	
1997 Q2	1				-0.3	
1997 Q3	2				-0.7	
1997 Q4	3		9.8	0.0	-0.5	
1998 Q1	4	11.8	9.9	0.0	1.5	11.3
1998 Q2	5	9.3	9.9	0.0	-0.3	9.6
1998 Q3	6	8.6	9.7	-0.0	-0.7	9.1
1998 Q4	7	9.3	9.8	0.0	-0.5	9.1
⋮	⋮	⋮	⋮	⋮	⋮	⋮
2017 Q1	80	12.4	10.9	0.1	1.5	12.5
2017 Q2	81	10.5	10.9	0.1	-0.3	10.5
2017 Q3	82	10.5	11.0	0.1	-0.7	10.1
2017 Q4	83	11.2	11.3	0.1	-0.5	10.4
	h					$\hat{y}_{T+h\|T}$
2018 Q1	1					12.9
2018 Q2	2					11.2
2018 Q3	3					11.0
2018 Q4	4					11.2
2019 Q1	5					13.4
2019 Q2	6					11.7
2019 Q3	7					11.5

Quarter	Time	Observation	Level	Slope	Season	Forecast
2019 Q4	8					11.7
2020 Q1	9					13.9
2020 Q2	10					12.2
2020 Q3	11					11.9
2020 Q4	12					12.2

Table 8.4: Applying Holt-Winters' method with multiplicative seasonality for forecasting domestic tourism in Australia. Notice that the multiplicative seasonal component sums to approximately $m = 4$. The smoothing parameters are $\alpha = 0.239$, $\beta^* = 0.0274$, $\gamma = 0.0008$ and $\mathrm{RMSE} = 0.412$.

Quarter	Time	Observation	Level	Slope	Season	Forecast
	t	y_t	l_t	b_t	s_t	$\hat{y}_{t+1\|t}$
1997 Q1	0				1.2	
1997 Q2	1				1.0	
1997 Q3	2				0.9	
1997 Q4	3		10.0	-0.0	0.9	
1998 Q1	4	11.8	10.0	-0.0	1.2	11.3
1998 Q2	5	9.3	9.9	-0.0	1.0	9.5
1998 Q3	6	8.6	9.8	-0.0	0.9	9.0
1998 Q4	7	9.3	9.8	-0.0	0.9	9.0
⋮	⋮	⋮	⋮	⋮	⋮	⋮
2017 Q1	80	12.4	10.8	0.1	1.2	12.5
2017 Q2	81	10.5	10.9	0.1	1.0	10.5
2017 Q3	82	10.5	11.1	0.1	0.9	10.1
2017 Q4	83	11.2	11.3	0.1	0.9	10.4
	h					$\hat{y}_{T+h\|T}$
2018 Q1	1					13.2
2018 Q2	2					11.1
2018 Q3	3					10.8
2018 Q4	4					11.1
2019 Q1	5					13.8

Quarter	Time	Observation	Level	Slope	Season	Forecast
2019 Q2	6					11.6
2019 Q3	7					11.2
2019 Q4	8					11.5
2020 Q1	9					14.3
2020 Q2	10					12.0
2020 Q3	11					11.6
2020 Q4	12					11.9

To compute the RMSE, we used the `evaluate()` and `rmse()` functions from `utilsforecast`. Because both methods have exactly the same number of parameters to estimate, we can compare the training RMSE from both models. In this case, the additive method fits the data as well as the multiplicative method.

```
fitted_vals = sf.forecast_fitted_values()
evaluate(fitted_vals, metrics=[rmse])
```

	unique_id	metric	additive	multiplicative
0	Holiday	rmse	0.412	0.412

To save and extract the parameters, we used `sf.fit()`.

```
sf.fit(aus_holidays)
for model in sf.fitted_[0]:
    print(model)
    # parameter layout: [alpha, beta, gamma, phi, l0, (b0 if trend), s1, ..., s
    par = model.model_["par"]
    components = model.model_["components"]
    trendtype = components[1]
    alpha, beta, gamma = np.round(par[:3], 4)
    level_0 = np.round(par[4], 4)
    seasonal_start = 4 + 1 + (1 if trendtype != "N" else 0)
    seasonal = np.round(par[seasonal_start:], 4)
    print("  Optimal parameters:")
    print("  alpha:", alpha)
    print("  beta:", beta)
    print("  gamma:", gamma)
    print("  Initial level:", level_0)
    print("  Initial seasonal components:", seasonal)
    print("  Sum of seasonal components:", seasonal.sum())
```

```
additive
  Optimal parameters:
```

```
  alpha: 0.2325
  beta: 0.03
  gamma: 0.0001
  Initial level: 9.8372
  Initial seasonal components: [-0.5386 -0.6916 -0.3051  1.5353]
  Sum of seasonal components: 2.220446049250313e-16
multiplicative
  Optimal parameters:
  alpha: 0.239
  beta: 0.0274
  gamma: 0.0008
  Initial level: 9.8161
  Initial seasonal components: [0.9429 0.9265 0.9688 1.1618]
  Sum of seasonal components: 4.0
```

The estimated components for both models are plotted in Figure 8.8.

```python
titles = [
    "ETS(A,A,A) decomposition\nAdditive Seasonality",
    "ETS(M,A,M) decomposition\nMultiplicative Seasonality",
]
states = ["Level", "Slope", "Season"]

fig, all_axes = plt.subplots(5, 2, figsize=(8, 8*14/12), sharex=True)
for (axes, title, model) in zip(all_axes.T, titles, sf.fitted_[0]):
    states = model.model_["states"]
    df = pd.DataFrame({
        "Trips": aus_holidays["y"].to_numpy(),
        "Level": states[:, 0][:-1],
        "Slope": states[:, 1][:-1],
        "Season": states[:, 2][:-1],
        "Residual": model.model_["residuals"],
        "Quarter": aus_holidays["ds"].to_numpy(),
    }).set_index("Quarter")
    for (ax, column) in zip(axes, df.columns):
        df[column].plot(ax=ax, ylabel=column)
    axes[0].set(title=title)
```

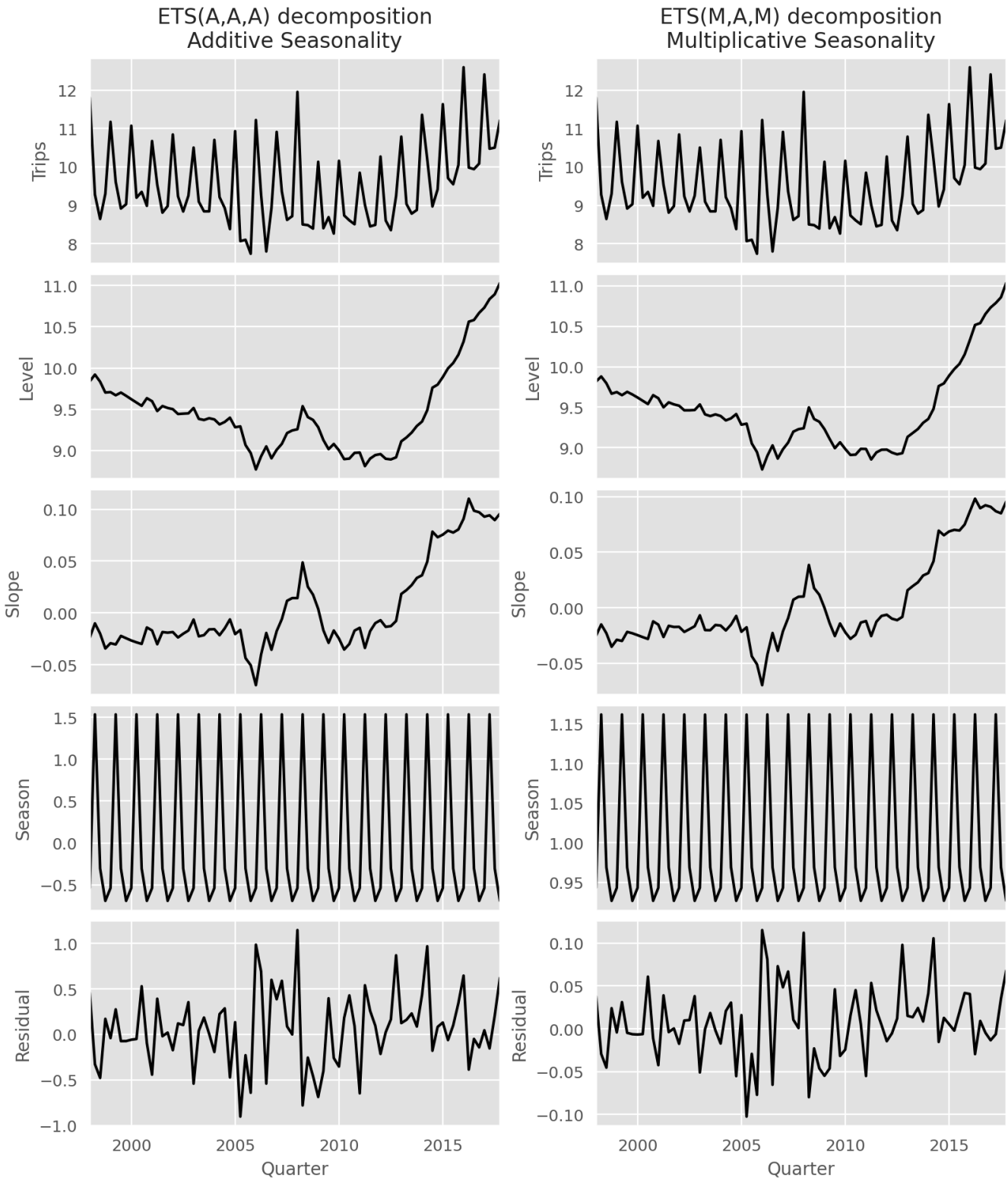

Figure 8.8: Estimated components for the Holt-Winters method with additive and multiplicative seasonal components.

Holt-Winters' damped method

Damping is possible with both additive and multiplicative Holt-Winters' methods. A method that often provides accurate and robust forecasts for seasonal data is the Holt-Winters method

with a damped trend and multiplicative seasonality:

$$
\begin{aligned}
\hat{y}_{t+h|t} &= \left[\ell_t + (\phi + \phi^2 + \cdots + \phi^h) b_t\right] s_{t+h-m(k+1)} \\
\ell_t &= \alpha(y_t / s_{t-m}) + (1-\alpha)(\ell_{t-1} + \phi b_{t-1}) \\
b_t &= \beta^*(\ell_t - \ell_{t-1}) + (1-\beta^*)\phi b_{t-1} \\
s_t &= \gamma \frac{y_t}{(\ell_{t-1} + \phi b_{t-1})} + (1-\gamma) s_{t-m}.
\end{aligned}
$$

Example: Holt-Winters method with daily data

The Holt-Winters method can also be used for daily type of data, where the seasonal period is $m = 7$, and the appropriate unit of time for h is in days. Here we forecast pedestrian traffic at a busy Melbourne train station in July 2016.

```
sth_cross_ped = (
    pd.read_csv("../data/pedestrian.csv", parse_dates=["ds"])
    .loc[lambda x:
        (x["unique_id"] == "Southern Cross Station")
        & (x["ds"].between("2016-07", "2016-08-14"))
    ]
    .groupby(["unique_id", "ds"])
    ["y"].sum().div(1e3)
    .reset_index()
)
train = sth_cross_ped.loc[lambda x: x["ds"] < "2016-08"]
sf = StatsForecast(
    models=[AutoETS(season_length=7, model="MAM",
        damped=True, alias="HW Damped")],
    freq="D",
)
level = [80, 95]
fc = sf.forecast(df=train, h=14, level=level)
plot_series(sth_cross_ped, fc, level=level,
    xlabel="Date", ylabel="Pedestrians ('000)",
    title="Daily traffic: Southern Cross",
    palette="black_and_blue", rm_legend=False)
```

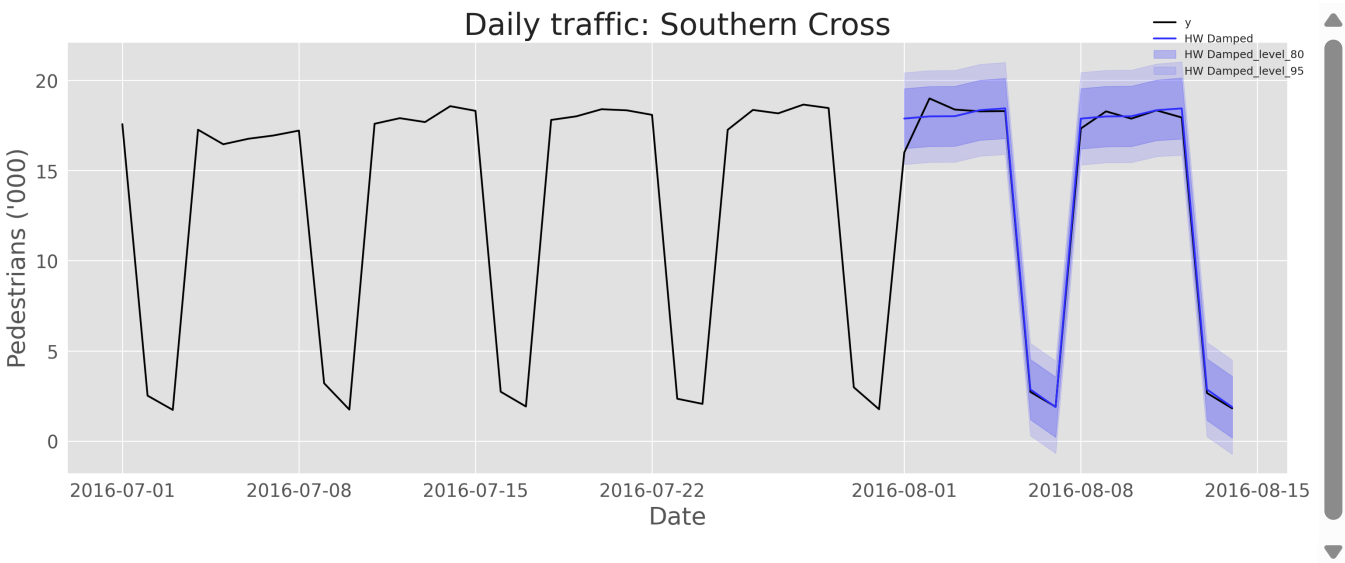

Figure 8.9: Forecasts of daily pedestrian traffic at the Southern Cross railway station, Melbourne.

Clearly the model has identified the weekly seasonal pattern and the increasing trend at the end of the data, and the forecasts are a close match to the test data.

8.4 A taxonomy of exponential smoothing methods

Exponential smoothing methods are not restricted to those we have presented so far. By considering variations in the combinations of the trend and seasonal components, nine exponential smoothing methods are possible, listed in Table 8.5. Each method is labelled by a pair of letters (T,S) defining the type of 'Trend' and 'Seasonal' components. For example, (A,M) is the method with an additive trend and multiplicative seasonality; (A_d,N) is the method with damped trend and no seasonality; and so on.

Table 8.5: A two-way classification of exponential smoothing methods.

Trend Component	Seasonal Component: N (None)	Seasonal Component: A (Additive)	Seasonal Component: M (Multiplicative)
N (None)	(N,N)	(N,A)	(N,M)
A (Additive)	(A,N)	(A,A)	(A,M)
A_d (Additive damped)	(A_d, N)	(A_d, A)	(A_d, M)

Some of these methods we have already seen using other names:

Table 8.6: Some common exponential smoothing methods.

Short hand	Method
(N, N)	Simple exponential smoothing
(A, N)	Holt's linear method
(A_d, N)	Additive damped trend method
(A, A)	Additive Holt-Winters' method
(A, M)	Multiplicative Holt-Winters' method
(A_d, A)	Holt-Winters' damped method

This type of classification was first proposed by Pegels (1969), who also included a method with a multiplicative trend. It was later extended by Gardner (1985) to include methods with an additive damped trend and by Taylor (2003) to include methods with a multiplicative damped

trend. We do not consider the multiplicative trend methods in this book as they tend to produce poor forecasts. See Hyndman et al. (2008) for a more thorough discussion of all exponential smoothing methods.

Table 8.7 gives the recursive formulas for applying the nine exponential smoothing methods in Table 8.5. Each cell includes the forecast equation for generating h-step-ahead forecasts, and the smoothing equations for applying the method.

Table 8.7: Formulas for recursive calculations and point forecasts. In each case, ℓ_t denotes the series level at time t, b_t denotes the slope at time t, s_t denotes the seasonal component of the series at time t, and m denotes the number of seasons in a year; α, β^*, γ and ϕ are smoothing parameters, $\phi_h = \phi + \phi^2 + \cdots + \phi^h$, and k is the integer part of $(h-1)/m$.

Trend	Seasonal		
	N	**A**	**M**
N	$\hat{y}_{t+h\|t} = \ell_t$ $\ell_t = \alpha y_t + (1-\alpha)\ell_{t-1}$	$\hat{y}_{t+h\|t} = \ell_t + s_{t+h-m(k+1)}$ $\ell_t = \alpha(y_t - s_{t-m}) + (1-\alpha)\ell_{t-1}$ $s_t = \gamma(y_t - \ell_{t-1}) + (1-\gamma)s_{t-m}$	$\hat{y}_{t+h\|t} = \ell_t s_{t+h-m(k+1)}$ $\ell_t = \alpha(y_t/s_{t-m}) + (1-\alpha)\ell_{t-1}$ $s_t = \gamma(y_t/\ell_{t-1}) + (1-\gamma)s_{t-m}$

8.5 Innovations state space models for exponential smoothing

In the rest of this chapter, we study the statistical models that underlie the exponential smoothing methods we have considered so far. The exponential smoothing methods presented in Table 8.7 are algorithms which generate point forecasts. The statistical models in this section generate the same point forecasts, but can also generate prediction (or forecast) intervals. A statistical model is a stochastic (or random) data generating process that can produce an entire forecast distribution. We will also describe how to use the model selection criteria introduced in Chapter 7 to choose the model in an objective manner.

Each model consists of a measurement equation that describes the observed data, and some state equations that describe how the unobserved components or states (level, trend, seasonal) change over time. Hence, these are referred to as **state space models**.

For each method there exist two models: one with additive errors and one with multiplicative errors. The point forecasts produced by the models are identical if they use the same smoothing

parameter values. They will, however, generate different prediction intervals.

To distinguish between a model with additive errors and one with multiplicative errors (and also to distinguish the models from the methods), we add a third letter to the classification of Table 8.5. We label each state space model as ETS$(\cdot,\cdot,\cdot)$ for (Error, Trend, Seasonal). This label can also be thought of as ExponenTial Smoothing. Using the same notation as in Table 8.5, the possibilities for each component (or state) are: $\text{Error} = \{A, M\}$, $\text{Trend} = \{N, A, A_d\}$ and $\text{Seasonal} = \{N, A, M\}$.

ETS(A,N,N): simple exponential smoothing with additive errors

Recall the component form of simple exponential smoothing:

$$\begin{aligned} \text{Forecast equation} \quad \hat{y}_{t+1|t} &= \ell_t \\ \text{Smoothing equation} \quad \ell_t &= \alpha y_t + (1-\alpha)\ell_{t-1}. \end{aligned}$$

If we re-arrange the smoothing equation for the level, we get the "error correction" form,

$$\begin{aligned} \ell_t &= \ell_{t-1} + \alpha(y_t - \ell_{t-1}) \\ &= \ell_{t-1} + \alpha e_t, \end{aligned}$$

where $e_t = y_t - \ell_{t-1} = y_t - \hat{y}_{t|t-1}$ is the residual at time t.

The training data errors lead to the adjustment of the estimated level throughout the smoothing process for $t = 1, \dots, T$. For example, if the error at time t is negative, then $y_t < \hat{y}_{t|t-1}$ and so the level at time $t-1$ has been over-estimated. The new level ℓ_t is then the previous level ℓ_{t-1} adjusted downwards. The closer α is to one, the "rougher" the estimate of the level (large adjustments take place). The smaller the α, the "smoother" the level (small adjustments take place).

We can also write $y_t = \ell_{t-1} + e_t$, so that each observation can be represented by the previous level plus an error. To make this into an innovations state space model, all we need to do is specify the probability distribution for e_t. For a model with additive errors, we assume that residuals (the one-step training errors) e_t are normally distributed white noise with mean 0 and variance σ^2. A short-hand notation for this is $e_t = \varepsilon_t \sim \text{NID}(0, \sigma^2)$; NID stands for "normally and independently distributed".

Then the equations of the model can be written as

$$\begin{aligned} y_t &= \ell_{t-1} + \varepsilon_t \\ \ell_t &= \ell_{t-1} + \alpha\varepsilon_t. \end{aligned} \tag{8.3}$$

We refer to the first line of these equations as the *measurement* (or observation) equation and the second line as the *state* (or transition) equation. These two equations, together with the

statistical distribution of the errors, form a fully specified statistical model. Specifically, these constitute an innovations state space model underlying simple exponential smoothing.

The term "innovations" comes from the fact that all equations use the same random error process, ε_t. For the same reason, this formulation is also referred to as a "single source of error" model. There are alternative multiple source of error formulations which we do not present here.

The measurement equation shows the relationship between the observations and the unobserved states. In this case, observation y_t is a linear function of the level ℓ_{t-1}, the predictable part of y_t, and the error ε_t, the unpredictable part of y_t. For other innovations state space models, this relationship may be nonlinear.

The state equation shows the evolution of the state through time. The influence of the smoothing parameter α is the same as for the methods discussed earlier. For example, α governs the amount of change in successive levels: high values of α allow rapid changes in the level; low values of α lead to smooth changes. If $\alpha = 0$, the level of the series does not change over time; if $\alpha = 1$, the model reduces to a random walk model, $y_t = y_{t-1} + \varepsilon_t$. (See Section 9.1 for a discussion of this model.)

ETS(M,N,N): simple exponential smoothing with multiplicative errors

In a similar fashion, we can specify models with multiplicative errors by writing the one-step-ahead training errors as relative errors

$$\varepsilon_t = \frac{y_t - \hat{y}_{t|t-1}}{\hat{y}_{t|t-1}}$$

where $\varepsilon_t \sim \text{NID}(0, \sigma^2)$. Substituting $\hat{y}_{t|t-1} = \ell_{t-1}$ gives $y_t = \ell_{t-1} + \ell_{t-1}\varepsilon_t$ and $e_t = y_t - \hat{y}_{t|t-1} = \ell_{t-1}\varepsilon_t$.

Then we can write the multiplicative form of the state space model as

$$\begin{aligned} y_t &= \ell_{t-1}(1 + \varepsilon_t) \\ \ell_t &= \ell_{t-1}(1 + \alpha\varepsilon_t). \end{aligned}$$

ETS(A,A,N): Holt's linear method with additive errors

For this model, we assume that the one-step-ahead training errors are given by $\varepsilon_t = y_t - \ell_{t-1} - b_{t-1} \sim \text{NID}(0, \sigma^2)$. Substituting this into the error correction equations for Holt's linear method we obtain

$$\begin{aligned} y_t &= \ell_{t-1} + b_{t-1} + \varepsilon_t \\ \ell_t &= \ell_{t-1} + b_{t-1} + \alpha\varepsilon_t \\ b_t &= b_{t-1} + \beta\varepsilon_t, \end{aligned}$$

where for simplicity we have set $\beta = \alpha\beta^*$.

ETS(M,A,N): Holt's linear method with multiplicative errors

Specifying one-step-ahead training errors as relative errors such that

$$\varepsilon_t = \frac{y_t - (\ell_{t-1} + b_{t-1})}{(\ell_{t-1} + b_{t-1})}$$

and following an approach similar to that used above, the innovations state space model underlying Holt's linear method with multiplicative errors is specified as

$$\begin{aligned} y_t &= (\ell_{t-1} + b_{t-1})(1 + \varepsilon_t) \\ \ell_t &= (\ell_{t-1} + b_{t-1})(1 + \alpha\varepsilon_t) \\ b_t &= b_{t-1} + \beta(\ell_{t-1} + b_{t-1})\varepsilon_t, \end{aligned}$$

where again $\beta = \alpha\beta^*$ and $\varepsilon_t \sim \text{NID}(0, \sigma^2)$.

Other ETS models

In a similar fashion, we can write an innovations state space model for each of the exponential smoothing methods of Table 8.7. Table 8.8 presents the equations for all of the models in the ETS framework.

Table 8.8: State space equations for each of the models in the ETS framework.

ADDITIVE ERROR MODELS

Trend	Seasonal

8.6 Estimation and model selection

Estimating ETS models

An alternative to estimating the parameters by minimising the sum of squared errors is to maximise the "likelihood". The likelihood is the probability of the data arising from the specified model. Thus, a large likelihood is associated with a good model. For an additive error model, maximising the likelihood (assuming normally distributed errors) gives the same results as minimising the sum of squared errors. However, different results will be obtained for multiplicative error models. In this section, we will estimate the smoothing parameters α, β, γ and ϕ, and the initial states ℓ_0, b_0, s_0, s_{-1}, $\ldots$, s_{-m+1}, by maximising the likelihood.

The possible values that the smoothing parameters can take are restricted. Traditionally, the parameters have been constrained to lie between 0 and 1 so that the equations can be interpreted as weighted averages. That is, $0 < \alpha, \beta^*, \gamma^*, \phi < 1$. For the state space models, we have set $\beta = \alpha\beta^*$ and $\gamma = (1-\alpha)\gamma^*$. Therefore, the traditional restrictions translate to $0 <$

$\alpha < 1, 0 < \beta < \alpha$ and $0 < \gamma < 1 - \alpha$. In practice, the damping parameter ϕ is usually constrained further to prevent numerical difficulties in estimating the model. In the `StatsForecast` package, it is restricted so that $0.8 < \phi < 0.98$.

Another way to view the parameters is through a consideration of the mathematical properties of the state space models. The parameters are constrained in order to prevent observations in the distant past having a continuing effect on current forecasts. This leads to some *admissibility* constraints on the parameters, which are usually (but not always) less restrictive than the traditional constraints region (Hyndman et al. 2008, 149–61). For example, for the ETS(A,N,N) model, the traditional parameter region is $0 < \alpha < 1$ but the admissible region is $0 < \alpha < 2$. For the ETS(A,A,N) model, the traditional parameter region is $0 < \alpha < 1$ and $0 < \beta < \alpha$ but the admissible region is $0 < \alpha < 2$ and $0 < \beta < 4 - 2\alpha$.

Model selection

A great advantage of the ETS statistical framework is that information criteria can be used for model selection. The AIC, AIC_c and BIC, introduced in Section 7.5, can be used here to determine which of the ETS models is most appropriate for a given time series.

For ETS models, Akaike's Information Criterion (AIC) is defined as

$$\text{AIC} = -2\log(L) + 2k,$$

where L is the likelihood of the model and k is the total number of parameters and initial states that have been estimated (including the residual variance).

The AIC corrected for small sample bias (AIC_c) is defined as

$$\text{AIC}_\text{c} = \text{AIC} + \frac{2k(k+1)}{T-k-1},$$

and the Bayesian Information Criterion (BIC) is

$$\text{BIC} = \text{AIC} + k[\log(T) - 2].$$

Three of the combinations of (Error, Trend, Seasonal) can lead to numerical difficulties. Specifically, the models that can cause such instabilities are ETS(A,N,M), ETS(A,A,M), and ETS(A,A_d,M), due to division by values potentially close to zero in the state equations. We normally do not consider these particular combinations when selecting a model.

Models with multiplicative errors are useful when the data are strictly positive, but are not numerically stable when the data contain zeros or negative values. Therefore, multiplicative error models will not be considered if the time series is not strictly positive. In that case, only the six fully additive models will be applied.

Example: Domestic holiday tourist visitor nights in Australia

We now employ the ETS statistical framework to forecast Australian holiday tourism over the period 2016–2019. We let `AutoETS()` select the model by minimising the AICc (this is done by default).

```
sf = StatsForecast(models=[AutoETS(season_length=4)], freq="Q")
sf.fit(aus_holidays)
autoets = sf.fitted_[0, 0]
model = autoets.model_
model["method"]
```

```
'ETS(M,N,M)'
```

The model selected is ETS(M,N,M):

$$
\begin{aligned}
y_t &= \ell_{t-1} s_{t-m}(1+\epsilon_t) \\
\ell_t &= \ell_{t-1}(1+\alpha\epsilon_t) \\
s_t &= s_{t-m}(1+\gamma\epsilon_t)
\end{aligned}
$$

```
params = np.round(autoets.model_["fit"].x, 4)
print("Optimal parameters:")
print("alpha:", params[0])
print("gamma:", params[1])
print("Initial level for series:", params[2])
print("Initial seasonal components:", params[3:])
```

```
Optimal parameters:
alpha: 0.3612
gamma: 0.0001
Initial level for series: 9.7878
Initial seasonal components: [0.9432 0.9268 0.9683]
```

The parameter estimates are $\hat{\alpha} = 0.3612$ and $\hat{\gamma} = 0.0001$. The output also returns the estimates for the initial states ℓ_0 and s_0, s_1, s_2 and s_3. Compare these with the values obtained for the Holt-Winters method with additive seasonality presented in Table 8.3.

Figure 8.10 shows the estimated components of the model, while Figure 8.12 shows point forecasts and prediction intervals generated from the model. The small values of γ indicate that the seasonal states change very little over time.

```
states = model["states"]
df = pd.DataFrame({
    "Trips": aus_holidays["y"].to_numpy(),
    "Level": states[:, 0][:-1],
    "Season": states[:, 2][:-1],
    "Residual": model["residuals"],
    "Quarter": aus_holidays["ds"].to_numpy(),
}).set_index("Quarter")
fig, axes = plt.subplots(4, 1, sharex=True)
for (ax, column) in zip(axes, df.columns):
```

```
    df[column].plot(ax=ax, ylabel=column)
axes[0].set(title="ETS(M,N,M) components")
plt.show()
```

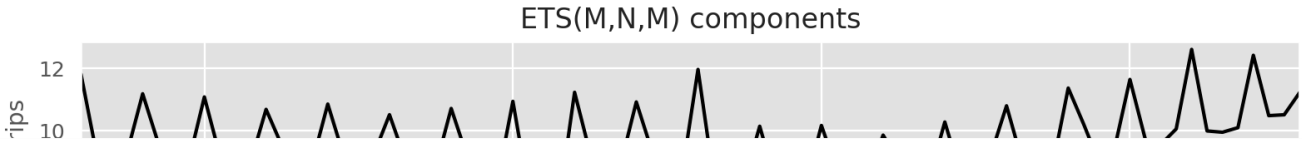

Figure 8.10: Graphical representation of the estimated states over time.

Because this model has multiplicative errors, the innovation residuals are not equivalent to the regular residuals (i.e., the one-step training errors). The innovation residuals are given by $\hat{\epsilon}_t$, while the regular residuals are defined as $y_t - \hat{y}_{t|t-1}$. They are plotted in Figure 8.11.

```
df = pd.DataFrame({
    "Innovation residuals": autoets.model_["residuals"],
    "Regular residuals": autoets.model_["actual_residuals"],
    "Quarter": aus_holidays["ds"].to_numpy(),
}).set_index("Quarter")
fig, axes = plt.subplots(2, 1, sharex=True)
for (ax, column) in zip(axes, df.columns):
    df[column].plot(ax=ax, ylabel=column)
plt.show()
```

Figure 8.11: Residuals and one-step forecast errors from the ETS(M,N,M) model.

8.7 Forecasting with ETS models

Point forecasts can be obtained from the models by iterating the equations for $t = T + 1, \dots, T + h$ and setting all $\varepsilon_t = 0$ for $t > T$.

For example, for model ETS(M,A,N), $y_{T+1} = (\ell_T + b_T)(1 + \varepsilon_{T+1})$. Therefore $\hat{y}_{T+1|T} = \ell_T + b_T$. Similarly,

$$\begin{aligned} y_{T+2} &= (\ell_{T+1} + b_{T+1})(1 + \varepsilon_{T+2}) \\ &= \left[(\ell_T + b_T)(1 + \alpha\varepsilon_{T+1}) + b_T + \beta(\ell_T + b_T)\varepsilon_{T+1}\right](1 + \varepsilon_{T+2}). \end{aligned}$$

Therefore, $\hat{y}_{T+2|T} = \ell_T + 2b_T$, and so on. These forecasts are identical to the forecasts from Holt's linear method, and also to those from model ETS(A,A,N). Thus, the point forecasts obtained from the method and from the two models that underlie the method are identical (assuming that the same parameter values are used). ETS point forecasts constructed in this way are equal to the means of the forecast distributions, except for the models with multiplicative seasonality (Hyndman et al. 2008).

To obtain forecasts from an ETS model, we use the `forecast()` method from the `StatsForecast` package. This function will always return the means of the forecast distribution, even when they differ from these traditional point forecasts.

```
sf = StatsForecast(models=[AutoETS(season_length=4)], freq="Q")
fc = sf.forecast(df=aus_holidays, h=8, level=[80, 95])
plot_series(aus_holidays, fc, level=[80, 95],
    xlabel="Quarter", ylabel="Overnight trips (millions)",
    title="Australian domestic tourism",
    palette="black_and_blue", rm_legend=False)
```

Figure 8.12: Forecasting Australian domestic overnight trips using an ETS(M,N,M) model.

Prediction intervals

A big advantage of the statistical models is that prediction intervals can also be generated — something that cannot be done using the point forecasting methods alone. The prediction intervals will differ between models with additive and multiplicative methods.

For most ETS models, a prediction interval can be written as

$$\hat{y}_{T+h|T} \pm c\sigma_h$$

where c depends on the coverage probability, and σ_h^2 is the forecast variance. Values for c were given in Table 5.1. For ETS models, formulas for σ_h^2 can be complicated; the details are given in Chapter 6 of Hyndman et al. (2008). In Table 8.9 we give the formulas for the additive ETS models, which are the simplest.

Table 8.9: Forecast variance expressions for each additive state space model, where σ^2 is the residual variance, m is the seasonal period, and k is the integer part of $(h-1)/m$ (i.e., the number of complete years in the forecast period prior to time $T+h$).

For a few ETS models, there are no known formulas for prediction intervals. In these cases, the `forecast()` method of the `StatsForecast` class uses simulated future sample paths and computes prediction intervals from the percentiles of these simulated future paths.

8.8 Exercises

1. Consider the number of pigs slaughtered in Victoria, available in the `aus_livestock` dataset.

   a. Use `AutoETS()` to estimate the equivalent model for simple exponential smoothing. Find the optimal values of α and ℓ_0, and generate forecasts for the next four months.
   b. Compute a 95% prediction interval for the first forecast using $\hat{y} \pm 1.96s$ where s is the standard deviation of the residuals. Compare your interval with the interval produced by `StatsForecast`.

2. Write your own function to implement simple exponential smoothing. The function should take arguments `y` (the time series), `alpha` (the smoothing parameter α) and `level` (the initial level ℓ_0). It should return the forecast of the next observation in the series. Does it give the same forecast as `SimpleExponentialSmoothing` from `StatsForecast`? Note that in `SimpleExponentialSmoothing`, you must specify the `alpha` parameter.

3. Modify your function from the previous exercise to return the sum of squared errors rather than the forecast of the next observation. Then find the optimal values of α and ℓ_0. Do you get the same values as `AutoETS(model='ANN')`?

4. Combine your previous two functions to produce a function that both finds the optimal values of α and ℓ_0, and produces a forecast of the next observation in the series.

5. Data set `global_economy` contains the annual Exports from many countries. Select one country to analyse.

   a. Plot the Exports series and discuss the main features of the data.
   b. Use an ETS(A,N,N) model to forecast the series, and plot the forecasts.
   c. Compute the RMSE values for the training data.
   d. Compare the results to those from an ETS(A,A,N) model. (Remember that the trended model is using one more parameter than the simpler model.) Discuss the merits of the two forecasting methods for this data set.
   e. Compare the forecasts from both methods. Which do you think is best?
   f. Calculate a 95% prediction interval for the first forecast for each model, using the RMSE values and assuming normal errors. Compare your intervals with those produced using `StatsForecast`.

6. Forecast the Chinese GDP from the `chinese_economy` data set using an ETS model. Note that the values for the GDP have already been transformed with a division by 10^6 for ease of use. Experiment with the various `AutoETS()` options to see how much the forecasts change with damped trend, or with a Box-Cox transformation. Try to develop an intuition of what each is doing to the forecasts.

[Hint: use a relatively large value of `h` when forecasting, so you can clearly see the differences between the various options when plotting the forecasts.]

7. Find an ETS model for the Gas data from `aus_production` and forecast the next few years. Why is multiplicative seasonality necessary here? Experiment with making the trend damped. Does it improve the forecasts?

8. Recall your retail time series data (from Exercise 7 in Section 2.10).

   a. Why is multiplicative seasonality necessary for this series?
   b. Apply Holt-Winters' multiplicative method to the data. Experiment with making the trend damped.
   c. Compare the RMSE of the one-step forecasts from the two methods. Which do you prefer?
   d. Check that the residuals from the best method look like white noise.
   e. Now find the test set RMSE, while training the model to the end of 2010. Can you beat the seasonal naïve approach from Exercise 7 in Section 5.11?

9. For the same retail data, try an STL decomposition applied to the Box-Cox transformed series, followed by ETS on the seasonally adjusted data. How does that compare with your best previous forecasts on the test set?

10. Compute the total domestic overnight trips across Australia from the `tourism` dataset.

    a. Plot the data and describe the main features of the series.
    b. Decompose the series using MSTL and obtain the seasonally adjusted data.
    c. Forecast the next two years of the series using an additive damped trend method applied to the seasonally adjusted data. (This can be specified using `damped=True`.)
    d. Forecast the next two years of the series using an appropriate model for Holt's linear method applied to the seasonally adjusted data (as before but without damped trend).
    e. Now use `AutoETS()` to choose a seasonal model for the data.
    f. Compare the RMSE of the ETS model with the RMSE of the models you obtained using MSTL decomposition. Which gives the better in-sample fits?
    g. Compare the forecasts from the three approaches. Which seems most reasonable?
    h. Check the residuals of your preferred model.

11. For this exercise use the quarterly number of arrivals to Australia from New Zealand, 1981 Q1 – 2012 Q3, from data set `aus_arrivals`.

    a. Make a time plot of your data and describe the main features of the series.

    b. Create a training set that withholds the last two years of available data. Forecast the test set using an appropriate model for Holt-Winters' multiplicative method.

    c. Why is multiplicative seasonality necessary here?

d. Forecast the two-year test set using each of the following methods:

- an ETS model;
- an additive ETS model applied to a log transformed series;
- a seasonal naïve method;
- a MSTL decomposition applied to the log transformed data followed by an ETS model applied to the seasonally adjusted (transformed) data.

e. Which method gives the best forecasts? Does it pass the residual tests?

f. Compare the same four methods using time series cross-validation instead of using a training and test set. Do you come to the same conclusions?

12. a. Apply cross-validation techniques to produce 1 year ahead ETS and seasonal naïve forecasts for Portland cement production (from `aus_production`). Use a stretching data window with initial size of 5 years, and increment the window by one observation.
    b. Compute the MSE of the resulting 4-step-ahead errors. Comment on which forecasts are more accurate. Is this what you expected?
13. Compare `AutoETS()` and `SeasonalNaive()` from `StatsForecast` on the following five time series. Use a test set of three years to decide what gives the best forecasts.
    - Beer and bricks production from `aus_production`.
    - Cost of drug subsidies for diabetes (`ATC2 == "A10"`) and corticosteroids (`ATC2 == "H02"`) from `pbs`.
    - Total food retailing turnover for Australia from `aus_retail`.
14. a. Use `AutoETS()` to select an appropriate model for the following series: total number of trips across Australia using `tourism`, the closing prices for the four stocks in `gafa_stock`, and the lynx series in `pelt`. Does it always give good forecasts?
    b. Find an example where it does not work well. Can you figure out why?
15. Show that the point forecasts from an ETS(M,A,M) model are the same as those obtained using Holt-Winters' multiplicative method.
16. Show that the forecast variance for an ETS(A,N,N) model is given by

$$\sigma^2 \left[1 + \alpha^2 (h-1)\right].$$

17. Write down 95% prediction intervals for an ETS(A,N,N) model as a function of ℓ_T, α, h and σ, assuming normally distributed errors.

8.9 Further reading

- Dois artigos de Ev Gardner (Gardner 1985 , 2006) fornecem uma ótima visão geral da história do alisamento exponencial e suas muitas variações.
- Um tratamento completo do assunto, fornecendo os detalhes matemáticos, é dado por Hyndman et al. (2008) .

8.10 Módulos e classes utilizados

Previsão de Estatísticas

- `StatsForecast` Classe - Mecanismo central de previsão para ajuste e geração de previsões
- `AutoETS()` Modelo - Para suavização exponencial automática, incluindo os métodos SES e de Holt.

Previsão de utilidades

- `evaluate()` Função - Para avaliar a precisão da previsão
- `plot_series()` Função - Para visualizar dados de séries temporais e previsões.
- `rmse()`, `mae()`, `mape()`, `mase()` métricas - Para calcular as métricas de precisão da previsão

---

1. Em alguns livros, é chamado de “suavização exponencial simples” .

2. Nossa implementação utiliza a estimação de máxima verossimilhança, conforme descrito na Seção 8.6 , enquanto Holt e Winters originalmente minimizaram a soma dos erros quadráticos. Para sazonalidade multiplicativa, isso levará a estimativas de parâmetros ligeiramente diferentes.

Publicado pela OTexts usando Quarto . © 2026